POLSKO-JAPOŃSKA AKADEMIA TECHNIK KOMPUTEROWYCH

Wydział Informatyki
Filia w Gdańsku

Otrombke Natalia
Nr albumu s26828
Nazwa specjalizacji: Cyberbezpieczeństwo

Kasprowicz Oliwier
Nr albumu s30290
Nazwa specjalizacji: Cyberbezpieczeństwo

Gutowski Michał
Nr albumu s30572
Nazwa specjalizacji: Cyberbezpieczeństwo

Parkitny Miłosz
Nr albumu s30287
Nazwa specjalizacji: Cyberbezpieczeństwo

Wronisz Wojciech
Nr albumu s26999
Nazwa specjalizacji: Cyberbezpieczeństwo

RacePortal — platforma wydarzeń motorsportowych

Rodzaj pracy
inżynierska

Imię i nazwisko promotora
dr hab. Marek Bendarczyk

Gdańsk, miesiąc, rok obrony

Streszczenie: Praca przedstawia projekt i implementację systemu RacePortal — platformy wspierającej katalogowanie wydarzeń motorsportowych oraz proces zapisów uczestników. System obejmuje aplikację webową, backend REST oparty o Spring Boot i MySQL oraz aplikację mobilną Expo pełniącą rolę publicznej wizytówki katalogu. W pracy opisano problem dziedzinowy, wymagania, architekturę, realizację kluczowych modułów (wydarzenia, garaż, zgłoszenia, role), mechanizmy bezpieczeństwa (JWT, RBAC) oraz weryfikację testami automatycznymi. Wynikiem jest działające, konteneryzowane środowisko demonstracyjne zgodne z zakresem MVP projektu.
Słowa kluczowe: motorsport, katalog wydarzeń, system zapisów, zarządzanie wydarzeniami, aplikacja mobilna

POLSKO-JAPOŃSKA AKADEMIA TECHNIK KOMPUTEROWYCH

Karta projektu

Temat projektu: RacePortal — platforma wydarzeń motorsportowych **Temat projektu po angielsku:** RacePortal — a platform for motorsports events	**Akronim:** RP **Data ustalenia tematu** 2025-10-01
Promotor: dr hab. Marek Bendarczyk	**Konsultanci:** 1. — brak —
Cele projektu: Celem projektu jest zaprojektowanie i zaimplementowanie systemu RacePortal — centralnego katalogu wydarzeń motorsportowych w Polsce, umożliwiającego przeglądanie wydarzeń (lista, mapa, kalendarz), zarządzanie garażem pojazdów, zgłoszenia udziału oraz obsługę ról organizatora i administratora, wraz z aplikacją mobilną typu wizytówka oraz konteneryzowanym środowiskiem uruchomieniowym.	
Rezultaty projektu: Aplikacja webowa (React/Vite), API REST (Spring Boot) z bazą MySQL, aplikacja mobilna Expo (katalog wydarzeń z CTA do weba), wspólne typy API (`packages/api-types`), dokumentacja projektowa oraz zestaw testów automatycznych (JUnit, Vitest, Playwright). Katalog wydarzeń z filtrami oraz widokami lista/mapa/kalendarz; szczegóły wydarzenia i wyznaczanie trasy (OSRM); konta użytkowników (JWT, RBAC); garaż pojazdów i dopasowanie kategorii do wydarzenia (`carMatch`); system zgłoszeń ze statusami i powiadomieniami e-mail; panele organizatora i administratora; archiwum; mobilna wizytówka z przekierowaniem zapisu na web.	
Miary sukcesu: Działający stos Docker Compose (web, API, MySQL, backup) z SMTP Mailbux; realizacja funkcji MVP katalogu, zapisów i ról; pozytywne wyniki testów automatycznych API/web/mobile unit; możliwość demonstracji pełnego scenariusza: gość → kierowca → organizator → administrator oraz mobilka → CTA web.	
Ograniczenia: Brak natywnej bramki płatności (PSP); upload plików zastąpiony adresami URL; aplikacja mobilna bez pełnej parity z webem (świadoma wizytówka); brak publikacji w sklepach App Store/Google Play; brak formalnego testu obciążenia 10k rekordów / 50 RPS; produkcyjny TLS poza domkniętym zakresem lokalnego MVP (w prod: Cloudflare Tunnel).	

Wykonawcy	**Numer albumu**	**Specjalizacja**	**Tryb studiów**
Otrombke Natalia	s26828	Cyberbezpieczeństwo	Stacjonarny
Kasprowicz Oliwier	s30290	Cyberbezpieczeństwo	Stacjonarny
Gutowski Michał	s30572	Cyberbezpieczeństwo	Stacjonarny
Parkitny Miłosz	s30287	Cyberbezpieczeństwo	Stacjonarny
Wronisz Wojciech	s26999	Cyberbezpieczeństwo	Stacjonarny

Data ukończenia projektu:	**Recenzent:**
7 listopada 2026	mgr Alex Polin

. . . . . .

(data, miejscowość)

OŚWIADCZENIE STUDENTA
DOTYCZĄCE WYKORZYSTANIA SZTUCZNEJ INTELIGENCJI (AI)

Ja, niżej podpisany/a: **Otrombke Natalia**

student/ka kierunku: **Cyberbezpieczeństwo**

numer albumu studenta: **s26828**

Oświadczam, że zapoznałem/am się z treścią Zarządzenia Rektora Polsko-Japońskiej Akademii Technik Komputerowych z dnia 1 kwietnia 2026 r. oraz z „Wytycznymi dotyczącymi wykorzystywania sztucznej inteligencji w procesie dydaktycznym PJATK", stanowiącymi załącznik do ww. zarządzenia.

Oświadczam, że ewentualne wykorzystanie narzędzi sztucznej inteligencji (AI) w niniejszej pracy miało charakter wyłącznie pomocniczy i nie stanowiło zastępstwa dla mojej samodzielnej pracy.

Oświadczam, że treść pracy stanowi moje własne, samodzielne opracowanie oraz ponoszę pełną odpowiedzialność za jej zawartość, niezależnie od wykorzystania narzędzi AI.

Oświadczam, że nie wykorzystywałem/am narzędzi AI do generowania treści stanowiących zasadniczą część pracy, w szczególności analiz, wniosków, interpretacji ani części podlegających ocenie jako przejaw twórczości własnej.

W przypadku korzystania z narzędzi AI sporządziłem/am Raport użycia AI zgodnie z obowiązującymi Wytycznymi i dołączyłem/am go do pracy.

. . . . . .

(czytelny podpis studenta/studentki)

. . . . . .

(data, miejscowość)

OŚWIADCZENIE STUDENTA
DOTYCZĄCE WYKORZYSTANIA SZTUCZNEJ INTELIGENCJI (AI)

Ja, niżej podpisany/a: **Kasprowicz Oliwier**

student/ka kierunku: **Cyberbezpieczeństwo**

numer albumu studenta: **s30290**

Oświadczam, że zapoznałem/am się z treścią Zarządzenia Rektora Polsko-Japońskiej Akademii Technik Komputerowych z dnia 1 kwietnia 2026 r. oraz z „Wytycznymi dotyczącymi wykorzystywania sztucznej inteligencji w procesie dydaktycznym PJATK", stanowiącymi załącznik do ww. zarządzenia.

Oświadczam, że ewentualne wykorzystanie narzędzi sztucznej inteligencji (AI) w niniejszej pracy miało charakter wyłącznie pomocniczy i nie stanowiło zastępstwa dla mojej samodzielnej pracy.

Oświadczam, że treść pracy stanowi moje własne, samodzielne opracowanie oraz ponoszę pełną odpowiedzialność za jej zawartość, niezależnie od wykorzystania narzędzi AI.

Oświadczam, że nie wykorzystywałem/am narzędzi AI do generowania treści stanowiących zasadniczą część pracy, w szczególności analiz, wniosków, interpretacji ani części podlegających ocenie jako przejaw twórczości własnej.

W przypadku korzystania z narzędzi AI sporządziłem/am Raport użycia AI zgodnie z obowiązującymi Wytycznymi i dołączyłem/am go do pracy.

. . . . . .

(czytelny podpis studenta/studentki)

. . . . . .

(data, miejscowość)

OŚWIADCZENIE STUDENTA
DOTYCZĄCE WYKORZYSTANIA SZTUCZNEJ INTELIGENCJI (AI)

Ja, niżej podpisany/a: **Gutowski Michał**

student/ka kierunku: **Cyberbezpieczeństwo**

numer albumu studenta: **s30572**

Oświadczam, że zapoznałem/am się z treścią Zarządzenia Rektora Polsko-Japońskiej Akademii Technik Komputerowych z dnia 1 kwietnia 2026 r. oraz z „Wytycznymi dotyczącymi wykorzystywania sztucznej inteligencji w procesie dydaktycznym PJATK", stanowiącymi załącznik do ww. zarządzenia.

Oświadczam, że ewentualne wykorzystanie narzędzi sztucznej inteligencji (AI) w niniejszej pracy miało charakter wyłącznie pomocniczy i nie stanowiło zastępstwa dla mojej samodzielnej pracy.

Oświadczam, że treść pracy stanowi moje własne, samodzielne opracowanie oraz ponoszę pełną odpowiedzialność za jej zawartość, niezależnie od wykorzystania narzędzi AI.

Oświadczam, że nie wykorzystywałem/am narzędzi AI do generowania treści stanowiących zasadniczą część pracy, w szczególności analiz, wniosków, interpretacji ani części podlegających ocenie jako przejaw twórczości własnej.

W przypadku korzystania z narzędzi AI sporządziłem/am Raport użycia AI zgodnie z obowiązującymi Wytycznymi i dołączyłem/am go do pracy.

. . . . . .

(czytelny podpis studenta/studentki)

. . . . . .

(data, miejscowość)

OŚWIADCZENIE STUDENTA
DOTYCZĄCE WYKORZYSTANIA SZTUCZNEJ INTELIGENCJI (AI)

Ja, niżej podpisany/a: **Parkitny Miłosz**

student/ka kierunku: **Cyberbezpieczeństwo**

numer albumu studenta: **s30287**

Oświadczam, że zapoznałem/am się z treścią Zarządzenia Rektora Polsko-Japońskiej Akademii Technik Komputerowych z dnia 1 kwietnia 2026 r. oraz z „Wytycznymi dotyczącymi wykorzystywania sztucznej inteligencji w procesie dydaktycznym PJATK", stanowiącymi załącznik do ww. zarządzenia.

Oświadczam, że ewentualne wykorzystanie narzędzi sztucznej inteligencji (AI) w niniejszej pracy miało charakter wyłącznie pomocniczy i nie stanowiło zastępstwa dla mojej samodzielnej pracy.

Oświadczam, że treść pracy stanowi moje własne, samodzielne opracowanie oraz ponoszę pełną odpowiedzialność za jej zawartość, niezależnie od wykorzystania narzędzi AI.

Oświadczam, że nie wykorzystywałem/am narzędzi AI do generowania treści stanowiących zasadniczą część pracy, w szczególności analiz, wniosków, interpretacji ani części podlegających ocenie jako przejaw twórczości własnej.

W przypadku korzystania z narzędzi AI sporządziłem/am Raport użycia AI zgodnie z obowiązującymi Wytycznymi i dołączyłem/am go do pracy.

. . . . . .

(czytelny podpis studenta/studentki)

. . . . . .

(data, miejscowość)

OŚWIADCZENIE STUDENTA
DOTYCZĄCE WYKORZYSTANIA SZTUCZNEJ INTELIGENCJI (AI)

Ja, niżej podpisany/a: **Wronisz Wojciech**

student/ka kierunku: **Cyberbezpieczeństwo**

numer albumu studenta: **s26999**

Oświadczam, że zapoznałem/am się z treścią Zarządzenia Rektora Polsko-Japońskiej Akademii Technik Komputerowych z dnia 1 kwietnia 2026 r. oraz z „Wytycznymi dotyczącymi wykorzystywania sztucznej inteligencji w procesie dydaktycznym PJATK", stanowiącymi załącznik do ww. zarządzenia.

Oświadczam, że ewentualne wykorzystanie narzędzi sztucznej inteligencji (AI) w niniejszej pracy miało charakter wyłącznie pomocniczy i nie stanowiło zastępstwa dla mojej samodzielnej pracy.

Oświadczam, że treść pracy stanowi moje własne, samodzielne opracowanie oraz ponoszę pełną odpowiedzialność za jej zawartość, niezależnie od wykorzystania narzędzi AI.

Oświadczam, że nie wykorzystywałem/am narzędzi AI do generowania treści stanowiących zasadniczą część pracy, w szczególności analiz, wniosków, interpretacji ani części podlegających ocenie jako przejaw twórczości własnej.

W przypadku korzystania z narzędzi AI sporządziłem/am Raport użycia AI zgodnie z obowiązującymi Wytycznymi i dołączyłem/am go do pracy.

. . . . . .

(czytelny podpis studenta/studentki)

Spis treści

1 Informacje wstępne **23**
1.1 Cel i zakres pracy 23
1.2 Wizja rozwiązania 23
1.3 Struktura pracy 24

2 Słownik pojęć **25**

3 Opis problemu **26**
3.1 Charakterystyka rynku wydarzeń motorsportowych 26
3.2 Problemy związane z organizacją wydarzeń i zapisami uczestników 26
3.3 Problemy informacyjne po stronie kierowców i organizatorów 26
3.4 Uzasadnienie potrzeby stworzenia systemu RacePortal 26
3.5 Aspekty techniczne i organizacyjne problemu 27

4 Kontekst biznesowy **28**
4.1 Segmentacja użytkowników 28
4.2 Interesariusze systemu 29
4.3 Propozycja wartości systemu 30
4.4 Kanały komunikacji i dystrybucji 32
4.5 Kluczowe zasoby i działania 32
4.6 Model działania platformy 33
4.7 Realistyczność wdrożenia i dalsze perspektywy 33
4.8 Kontekst prawny i etyczny 34

5 Analiza SWOT **35**
5.1 Mocne strony (Strengths) 35
5.2 Słabe strony (Weaknesses) 35
5.3 Szanse (Opportunities) 35
5.4 Zagrożenia (Threats) 36

6 Planowanie projektu **37**
6.1 Cele projektu 37

6.2 Zakres projektu . . . 37
6.3 Metodyka pracy . . . 37
6.4 Organizacja zespołu . . . 37
6.5 Harmonogram pracy . . . 37
6.6 Infrastruktura i środowisko pracy . . . 38
6.7 Analiza ryzyka projektowego . . . 38
6.7.1 Analiza zagrożeń . . . 38

7 Analiza wymagań **39**
7.1 Źródła i sposób pozyskania wymagań . . . 39
7.2 Wymagania ogólne i dziedzinowe . . . 39
7.3 Wymagania funkcjonalne . . . 39
7.4 Wymagania niefunkcjonalne . . . 40
7.5 Wymagania bezpieczeństwa . . . 40
7.6 Wymagania interfejsu użytkownika . . . 41
7.7 Diagramy UML . . . 41
7.7.1 Diagram przypadków użycia . . . 41
7.7.2 Diagram sekwencji: rejestracja i logowanie . . . 41
7.7.3 Diagram sekwencji: zapis na wydarzenie . . . 41
7.7.4 Diagram sekwencji: zarządzanie garażem . . . 41

8 Projekt systemu **42**
8.1 Założenia architektoniczne . . . 42
8.2 Architektura logiczna systemu . . . 42
8.3 Architektura fizyczna i wdrożeniowa . . . 42
8.4 Projekt bazy danych . . . 42
8.4.1 Diagram klas . . . 42
8.5 Projekt interfejsu REST API . . . 43
8.6 Projekt warstwy webowej . . . 43
8.6.1 Prototyp UI w Figma Make . . . 43
8.7 Projekt aplikacji mobilnej . . . 44
8.8 Projekt mechanizmów bezpieczeństwa . . . 44

9 Realizacja **45**
9.1 Organizacja implementacji . . . 45
9.2 Podział systemu na komponenty . . . 45
9.2.1 Backend . . . 45
9.2.2 Frontend webowy . . . 45
9.2.3 Aplikacja mobilna . . . 45

9.3 Integracja komponentów 45
9.4 Konteneryzacja i uruchamianie środowiska 45

10 Szczegóły implementacji backendu **46**
10.1 Struktura projektu backendowego 46
10.2 Model domenowy i encje 46
10.3 Rejestracja użytkowników i logowanie 46
10.4 Zarządzanie profilami użytkowników 46
10.5 Obsługa wydarzeń 46
10.6 Obsługa zgłoszeń 47
10.7 Obsługa garażu 47
10.8 Panel administratora i organizatora 47
10.9 Integracja z bazą danych 47
10.10 Obsługa wyjątków i walidacja 47

11 Szczegóły implementacji frontendu webowego **48**
11.1 Struktura aplikacji webowej 48
11.2 Strona główna i katalog wydarzeń 48
11.3 Widok szczegółów wydarzenia 48
11.4 Logowanie i rejestracja 48
11.5 Dashboard użytkownika 48
11.6 Widok garażu 48
11.7 Panel organizatora 49
11.8 Panel administratora 49
11.9 Mapa i elementy lokalizacyjne 49
11.10 Responsywność i doświadczenie użytkownika 49

12 Szczegóły implementacji aplikacji mobilnej **50**
12.1 Założenia aplikacji mobilnej 50
12.2 Architektura rozwiązania mobilnego 50
12.3 Lista wydarzeń i widok szczegółów 50
12.4 Nawigacja i interfejs użytkownika 50
12.5 Integracja z API 50
12.6 Przekierowanie do wersji webowej 50
12.7 Ograniczenia wersji mobilnej 51

13 Bezpieczeństwo systemu **52**
13.1 Założenia bezpieczeństwa 52
13.2 Klasyfikacja danych 52
13.3 Uwierzytelnianie użytkowników 52

13.4 Autoryzacja i kontrola dostępu RBAC . . . 52
13.5 Wykorzystanie tokenów JWT . . . 52
13.6 Walidacja danych wejściowych . . . 52
13.7 Ochrona zasobów użytkowników . . . 52
13.8 Bezpieczeństwo komunikacji . . . 53
13.9 Analiza zagrożeń . . . 53
13.10 Analiza ryzyka . . . 53
13.11 Ograniczenia przyjętych zabezpieczeń . . . 53

14 Testowanie i weryfikacja systemu **54**
14.1 Strategia testowania . . . 54
14.2 Środowisko testowe . . . 54
14.3 Testy jednostkowe . . . 54
14.4 Testy integracyjne backendu . . . 54
14.5 Testy End-to-End aplikacji webowej . . . 54
14.6 Testy aplikacji mobilnej . . . 54
14.7 Testy mechanizmów bezpieczeństwa . . . 54
14.8 Automatyzacja uruchamiania testów . . . 55
14.9 Analiza wyników testów . . . 55
14.10 Ograniczenia procesu testowania . . . 55

15 Prezentacja systemu **56**
15.1 Prezentacja głównych funkcjonalności . . . 56
15.2 Scenariusz użycia z perspektywy kierowcy . . . 56
15.3 Scenariusz użycia z perspektywy organizatora . . . 56
15.4 Scenariusz użycia z perspektywy administratora . . . 56
15.5 Podsumowanie działania systemu . . . 56

16 Nakład pracy **57**
16.1 Liczba godzin przeznaczona na realizację projektu . . . 57
16.2 Podział zadań w zespole . . . 57
16.3 Narzędzia wspierające zarządzanie pracą . . . 57
16.4 Problemy organizacyjne i ich rozwiązanie . . . 57

17 Wkład własny **58**
17.1 Natalia Otrombke . . . 58
17.2 Oliwier Kasprowicz . . . 58
17.3 Michał Gutowski . . . 58
17.4 Miłosz Parkitny . . . 59
17.5 Wojciech Wronisz . . . 59

18 Podsumowanie **60**
18.1 Największe wyzwania i sposób ich rozwiązania 60
18.2 Zmiany i adaptacje w trakcie realizacji 60
18.3 Ograniczenia końcowej wersji systemu 60
18.3.1 Ograniczenia wydajnościowe (RPS) 60
18.4 Możliwości dalszego rozwoju 61
18.5 Wnioski końcowe 61

Bibliografia **62**

Spis rysunków **63**

Spis tabel **64**

Załączniki **65**

Rozdział 1

Informacje wstępne

1.1 Cel i zakres pracy

Celem pracy jest zaprojektowanie oraz implementacja systemu **RacePortal** — platformy internetowej (z uzupełnieniem mobilnym) służącej jako centralny katalog wydarzeń motorsportowych w Polsce oraz narzędzie do obsługi zapisów kierowców.

Zakres obejmuje:

- aplikację webową (katalog, konta, garaż, zgłoszenia, panele organizatora i administratora);
- API REST z trwałą bazą danych wydarzeń, użytkowników i zgłoszeń;
- powiadomienia e-mail (potwierdzenia, statusy, weryfikacja konta);
- mapę wydarzeń oraz wyznaczanie trasy dojazdu (OSRM; opcjonalnie Google Maps przy kluczu API);
- aplikację mobilną Expo jako *wizytówkę* katalogu z CTA zapisu na stronie webowej;
- konteneryzację (Docker Compose) oraz podstawowy zestaw testów automatycznych.

Poza zakresem MVP pozostają m.in. natywne płatności online, pełny upload plików binarnych, push notifications oraz publikacja aplikacji mobilnej w sklepach.

1.2 Wizja rozwiązania

RacePortal ma być **jednym miejscem** odkrywania i obsługi startów motorsportowych w Polsce: kierowca znajduje wydarzenie, sprawdza wymagania względem własnego garażu i składa zgłoszenie; organizator publikuje imprezę i zarządza listą uczestników; administrator moderuje treści i role.

Wizja produktowa zakłada:

- publiczny katalog z trzema widokami (lista, mapa, kalendarz) na wspólnych filtrach;
- ujednolicony proces zapisu ze statusami i powiadomieniami e-mail;
- wyraźny podział ról `USER` / `ORGANIZER` / `ADMIN` (gość przegląda bez konta);
- mobilną ścieżkę discovery (Expo) z konwersją zapisu na web;
- uruchamialne środowisko demonstracyjne (Docker) oraz wdrożenie produkcyjne za Cloudflare Tunnel.

Szczegóły warstw technicznych (web React/Vite, API Spring Boot, MySQL, mobile Expo, `packages/api-types`) opisano w rozdziale 8 oraz w rozdziałach implementacyjnych. Kod źródłowy utrzymywany jest w repozytorium projektu [1].

1.3 Struktura pracy

Kolejne rozdziały obejmują: słownik pojęć, opis problemu dziedzinowego, kontekst biznesowy (w tym karty udziałowców), analizę SWOT, planowanie projektu, analizę wymagań (w tym karty wymagań i diagramy UML), projekt systemu (m.in. Figma, diagram klas), opis realizacji oraz szczegóły implementacji backendu, frontendu i mobile, bezpieczeństwo, testowanie, prezentację systemu, nakład pracy, **wkład własny** oraz podsumowanie (z uwzględnieniem ograniczeń wydajnościowych RPS).

Rozdział 2

Słownik pojęć

Wydarzenie (event) Zorganizowany start / impreza motorsportowa (np. track day, drift, rally), opisana datą, lokalizacją, kategorią i wymaganiami.

Katalog Publiczny wykaz wydarzeń z widokami lista, mapa i kalendarz oraz filtrami.

Garaż Zbiór pojazdów przypisanych do konta kierowcy, wykorzystywanych przy zapisie na wydarzenie.

Zgłoszenie (registration) Wniosek o udział w wydarzeniu, posiadający status (np. PENDING, APPROVED, CANCELED).

carMatch Mechanizm dopasowania kategorii/klasy pojazdu do kategorii wydarzenia (aliasy nazw).

Organizator Użytkownik z rolą ORGANIZER, uprawniony do tworzenia i zarządzania własnymi wydarzeniami oraz zgłoszeniami.

Administrator Użytkownik z rolą ADMIN, moderujący wydarzenia, wnioski o rolę organizatora i role kont.

Wizytówka mobilna Wersja aplikacji Expo ograniczona do przeglądania katalogu; zapis odbywa się na webie (CTA).

JWT JSON Web Token — token sesji używany do uwierzytelniania żądań API.

RBAC Role-Based Access Control — kontrola dostępu oparta o role.

OSRM Open Source Routing Machine — publiczny serwis wyznaczania trasy wykorzystany jako domyślny provider.

MVP Minimum Viable Product — minimalny zakres funkcji przyjęty do odbioru projektu.

RPS Requests Per Second — metryka obciążenia (np. docelowy test 50 RPS przy 10k rekordach).

Rozdział 3

Opis problemu

3.1 Charakterystyka rynku wydarzeń motorsportowych

Amatorski i półprofesjonalny motorsport w Polsce obejmuje liczne formy aktywności (track day, drift, rally, wyścigi górskie itp.). Informacje o terminach i zapisach są często rozproszone pomiędzy strony organizatorów, media społecznościowe oraz komunikatory.

3.2 Problemy związane z organizacją wydarzeń i zapisami uczestników

Organizatorzy potrzebują narzędzia do publikacji wydarzenia, zbierania zgłoszeń, weryfikacji wymagań wobec pojazdu/kierowcy oraz komunikacji statusu zgłoszenia. Ręczne procesy (arkusze, wiadomości prywatne) są podatne na błędy i trudne do skalowania.

3.3 Problemy informacyjne po stronie kierowców i organizatorów

Kierowcy mają trudność ze znalezieniem aktualnych startów w jednym miejscu, porównaniem wymagań wydarzenia z posiadanym pojazdem oraz śledzeniem statusu zapisu. Organizatorzy — z utrzymaniem spójnej listy uczestników i powiadomień.

3.4 Uzasadnienie potrzeby stworzenia systemu RacePortal

RacePortal odpowiada na potrzebę **centralnego katalogu** oraz **ujednoliconego procesu zapisu** z rolami użytkowników, przy zachowaniu prostego uruchomienia demonstracyjnego (Docker) i ścieżki mobilnej (odkrywanie wydarzeń).

3.5 Aspekty techniczne i organizacyjne problemu

Wyzwania obejmują: spójny model danych wydarzeń i zgłoszeń, bezpieczeństwo kont (JWT, RBAC), walidację wymagań wydarzenia względem garażu, integrację mapy/trasy, dostarczalność maili w środowisku deweloperskim oraz świadomy podział odpowiedzialności web vs mobile.

Rozdział 4

Kontekst biznesowy

Niniejszy rozdział sytuuje RacePortal w kontekście produktowym i organizacyjnym: identyfikuje segmenty odbiorców, udziałowców oraz propozycję wartości, a następnie opisuje kanały dystrybucji, zasoby i działania kluczowe, model funkcjonowania platformy, realność wdrożenia oraz uwarunkowania prawno-etyczne. Punktem wyjścia są założenia Dokumentu Założeń Wstępnych (DZW), karty projektu, Business Model Canvas (BMC / materiały KMR) oraz scenariusze walidacji wywiadami, konfrontowane z faktyczną implementacją systemu (role RBAC, katalog wydarzeń, garaż, zgłoszenia, maile, wdrożenie produkcyjne).

Należy zaznaczyć, że segmentacja w materiałach KMR została sformułowana jako *założenie koncepcyjne*, a nie jako wynik reprezentatywnych badań rynkowych. Dokument *Interview Validation* definiuje hipotezy, skrypt wywiadu oraz metryki sukcesu; nie zawiera wyników przeprowadzonych wywiadów, dlatego w dalszej części nie są one przytaczane jako dane empiryczne.

4.1 Segmentacja użytkowników

Z punktu widzenia produktu wyróżnia się cztery podstawowe segmenty korzystające z systemu, odpowiadające poziomom dostępu zaimplementowanym w warstwie aplikacji. Segmentacja ta jest spójna z DZW (kategorie użytkowników MVP), z BMC (kierowcy, pasjonaci/kibice, organizatorzy) oraz z modelem bezpieczeństwa opartym o role.

Gość (użytkownik anonimowy) korzysta z publicznego katalogu bez uwierzytelnienia. W implementacji nie istnieje osobna rola `GUEST` — dostęp gościa wynika z konfiguracji publicznych ścieżek API (`permitAll` dla m.in. `/api/events/**`, `/api/tracks/**`, `/api/health`) oraz publicznych tras aplikacji webowej (strona główna, katalog, detal wydarzenia, archiwum, strony prawne). Segment ten obejmuje kibiców i pasjonatów poszukujących informacji o terminach i lokalizacjach, a także osoby rozważające rejestrację.

Kierowca odpowiada roli systemowej `USER`. Po utworzeniu konta użytkownik utrzymuje profil, zarządza garażem pojazdów oraz składa zgłoszenia udziału w wydarzeniach. Kluczo-

wą wartością dla tego segmentu jest możliwość dopasowania pojazdu do wymagań wydarzenia (mechanizm `carMatch` / `CategoryMatcher`) oraz śledzenie statusu zgłoszenia wraz z powiadomieniami e-mail. W materiałach KMR segment ten jest charakteryzowany demograficznie jako osoby w przybliżonym przedziale 16–40 lat, geograficznie zorientowane na rynek polski. W zaimplementowanym MVP aplikacja nie egzekwuje ograniczeń wiekowych po stronie rejestracji; kryterium pełnoletności pojawia się natomiast w skryningu respondentów dokumentu walidacyjnego wywiadów.

Organizator odpowiada roli `ORGANIZER`. Segment obejmuje mniejszych i większych organizatorów imprez (track day, drift, time attack, KJS, rajdy itd.), którzy potrzebują kanału publikacji wydarzeń oraz narzędzia do obsługi listy zgłoszeń. W systemie uprawnienie organizatora nie jest przyznawane automatycznie przy rejestracji: użytkownik składa wniosek o rolę, a administrator go akceptuje lub odrzuca. Po weryfikacji organizator tworzy i edytuje własne wydarzenia, zarządza statusami zgłoszeń oraz komunikuje się z uczestnikami poprzez mechanizmy platformy (w tym powiadomienia systemowe).

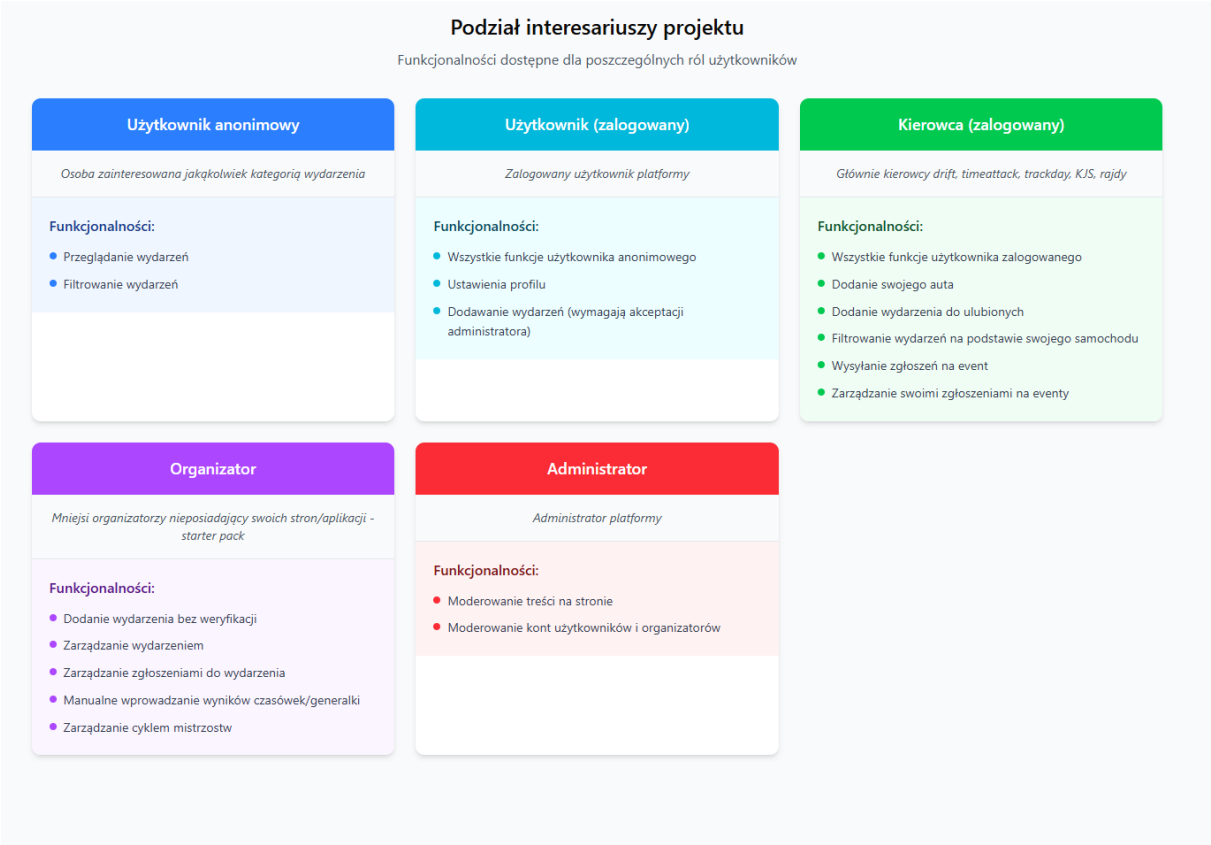
Administrator odpowiada roli `ADMIN`. Segment ten reprezentuje operatora platformy odpowiedzialnego za jakość katalogu: moderację wydarzeń oczekujących na publikację, zarządzanie rolami kont oraz rozstrzyganie wniosków o uprawnienia organizatora. W warstwie produktowej administrator realizuje model *human-in-the-loop* przyjęty w analizie etycznej projektu — ostateczne decyzje moderacyjne pozostają po stronie człowieka, a nie algorytmu.

Wczesny, koncepcyjny podział funkcjonalności według ról (materiał projektowy zespołu) przedstawia rys. 4.1. Należy go interpretować jako mapę intencji produktowej; szczegółowe mapowanie na wymagania i endpointy API opisano w rozdziałach 7–8.

4.2 Interesariusze systemu

Analiza udziałowców wyróżnia zarówno *aktorów systemowych* (role RBAC oraz gość), jak i szersze otoczenie opisane w dokumentacji SAI (m.in. zespół wytwórczy, media, właściciele obiektów, służby zabezpieczające imprezy, a także udziałowcy nieożywieni: baza danych oraz serwisy mapowe). W niniejszej pracy karty udziałowców ograniczono do trzech kluczowych odbiorców końcowych platformy, ponieważ to ich potrzeby determinują zakres MVP oraz podział uprawnień. Pozostali udziałowcy (np. właściciele torów, media) korzystają pośrednio z publicznego katalogu lub pozostają poza zaimplementowanym przepływem transakcyjnym.

Poniżej przedstawiono karty udziałowców w formacie środowiska `stakeholder` szablonu PJATK. Odwołania do wymagań i projektu wskazują rozdziały 7–8.



Rysunek 4.1: Podział interesariuszy projektu — funkcjonalności dostępne dla ról użytkowników (materiał koncepcyjny).

KARTA UDZIAŁOWCA	
Identyfikator:	U-01
Nazwa:	Kierowca (USER)
Opis:	Użytkownik z kontem, który utrzymuje garaż pojazdów i składa zgłoszenia na wydarzenia motorsportowe.
Typ udziałowca:	Użytkownik końcowy / klient platformy
Punkt widzenia:	Chce szybko znaleźć starty, dopasować auto do wymagań i śledzić status zgłoszenia.
Ograniczenia:	Zapis i garaż wymagają konta web; mobilka w MVP jest wizytówką bez pełnej parity.
Wymagania:	Wymagania funkcjonalne katalogu, garażu i zgłoszeń — rozdz. 7; panel użytkownika — rozdz. 11.

Tabela 4.1: Karta udziałowca: Kierowca

4.3 Propozycja wartości systemu

Propozycja wartości RacePortal wynika wprost z diagnozy problemu rozproszenia informacji o wydarzeniach motorsportowych w Polsce. Dotychczasowe praktyki użytkowników — grupy w mediach społecznościowych, witryny poszczególnych torów oraz statyczne kalendarze — nie zapewniają jednolitego procesu odkrywania imprez, weryfikacji wymagań wobec pojazdu

KARTA UDZIAŁOWCA	
Identyfikator:	U-02
Nazwa:	Organizator (ORGANIZER)
Opis:	Podmiot publikujący wydarzenia i zarządzający listą zgłoszeń uczestników.
Typ udziałowca:	Dostawca treści / partner biznesowy
Punkt widzenia:	Potrzebuje prostego CRUD wydarzeń, weryfikacji zgłoszeń i komunikacji statusów (e-mail).
Ograniczenia:	MVP bez natywnej bramki płatności (PSP); rozliczenie wpisowego poza platformą, z dowodem płatności (URL lub plik w magazynie uploadów).
Wymagania:	Panel organizatora i RBAC — rozdz. 7, 8; implementacja — rozdz. 10.

Tabela 4.2: Karta udziałowca: Organizator

KARTA UDZIAŁOWCA	
Identyfikator:	U-03
Nazwa:	Administrator (ADMIN)
Opis:	Operator platformy moderujący wydarzenia, role kont oraz wnioski o uprawnienia organizatora.
Typ udziałowca:	Właściciel / operator systemu
Punkt widzenia:	Zapewnia jakość katalogu i bezpieczeństwo ról bez ręcznej ingerencji w bazę.
Ograniczenia:	Skala produkcyjna (pełne procesy RODO, WAF, 2FA) poza zakresem MVP demonstracyjnego.
Wymagania:	Panel admina, bezpieczeństwo JWT/RBAC — rozdz. 13; testy RBAC — rozdz. 14.

Tabela 4.3: Karta udziałowca: Administrator

ani obsługi statusu zapisu. RacePortal odpowiada na ten deficyt jako **centralny katalog** oraz **warstwa operacyjna zapisów**, z rozdzielonymi narzędziami dla poszczególnych ról.

Dla kierowców wartość koncentruje się na: (1) przejrzystym katalogu z widokami lista, mapa i kalendarz oraz wspólnymi filtrami; (2) wirtualnym garażu pojazdów; (3) dopasowaniu pojazdu do kategorii/wymagań wydarzenia; (4) złożeniu zgłoszenia i otrzymywaniu powiadomień o statusie. Dla organizatorów wartość obejmuje publikację wydarzenia, zarządzanie listą zgłoszeń, weryfikację uczestników oraz komunikację statusów bez konieczności prowadzenia procesu wyłącznie w arkuszach i wiadomościach prywatnych. Dla gości (kibiców i pasjonatów) wartość sprowadza się do bezpłatnego, natychmiastowego dostępu do informacji o terminach i lokalizacjach, w tym do widoku mapy oraz trasy dojazdu (domyślnie OSRM; opcjonalnie Google Maps przy skonfigurowanym kluczu API).

Wyróżnikiem produktowym względem prostych kalendarzy branżowych jest integracja łańcucha *odkrycie* $\rightarrow$ *dopasowanie pojazdu* $\rightarrow$ *zgłoszenie* $\rightarrow$ *status*, a nie wyłącznie publikacja dat. Założenie to potwierdza dokumentacja walidacyjna: najistotniejszym ryzykiem biznesowym

jest odmowa migracji społeczności z grup społecznościowych do scentralizowanego narzędzia; bez aktualnej podaży wydarzeń katalog traci wartość dla kierowców (problem klasycznej dwustronnej platformy).

4.4 Kanały komunikacji i dystrybucji

Głównym kanałem dostępu do produktu jest aplikacja webowa dostępna pod adresem `https://raceportal.pl` (oraz wariant `www`). Interfejs webowy pełni rolę pełnego klienta transakcyjnego: rejestracja i logowanie, garaż, zgłoszenia, panele organizatora i administratora. Dodatkowo warstwa web została przygotowana jako PWA (manifest, service worker, nawigacja mobilna), co umożliwia instalację skrótu na ekranie początkowym urządzenia bez publikacji w sklepach aplikacyjnych.

Kanałem uzupełniającym discovery jest aplikacja mobilna Expo, zaprojektowana świadomie jako *wizytówka*: umożliwia przeglądanie katalogu i szczegółów wydarzenia, natomiast zapis udziału realizowany jest poprzez CTA otwierające odpowiednią stronę webową. Takie rozdzielenie ogranicza zakres MVP mobilnego i koncentruje złożoną logikę konta po stronie weba.

Kanałem komunikacji transakcyjnej jest poczta elektroniczna dostarczana przez SMTP dostawcy **Mailbux** (nadawca `noreply@raceportal.pl`). System wysyła m.in. wiadomości aktywacyjne i resetu hasła, potwierdzenia zgłoszeń, zmiany statusów, powiadomienia o anulowaniu lub zmianie wydarzenia oraz decyzje dotyczące ról i wniosków organizatora. Wczesny wariant deweloperski oparty o Mailpit został porzucony na rzecz realnej skrzynki domenowej, aby demonstracja rejestracji odpowiadała doświadczeniu użytkownika końcowego.

Kanałem dystrybucji kodu i współpracy zespołu pozostaje repozytorium GitHub [1]. W materiałach BMC przewidziano również promocję organiczną w mediach społecznościowych oraz współpracę partnerską z klubami; w fazie MVP działania te mają charakter organizacyjny i nie są zautomatyzowane w kodzie aplikacji.

4.5 Kluczowe zasoby i działania

Do zasobów kluczowych należą: pięcioosobowy zespół projektowy (specjalizacja cyberbezpieczeństwo), kod źródłowy monorepozytorium (warstwy web, API, mobile oraz pakiet wspólnych typów `packages/api-types`), relacyjna baza MySQL z seedem danych demonstracyjnych, infrastruktura kontenerowa Docker Compose oraz środowisko produkcyjne oparte o maszynę wirtualną i tunel Cloudflare. Zasobem produktowym jest również katalog torów polskich wykorzystywany przy tworzeniu i filtrowaniu wydarzeń.

Działania kluczowe obejmują: rozwój i utrzymanie platformy; zapewnienie uruchamialności środowiska (Compose lokalnie i na VM); moderację treści oraz weryfikację organizatorów;

utrzymanie mechanizmów bezpieczeństwa (JWT, RBAC, walidacja wejścia, kierunek zgodności z OWASP ASVS Level 1); utrzymanie dostarczalności powiadomień e-mail; a także dokumentowanie zmian i weryfikację testami automatycznymi. W ujęciu BMC działania marketingowe i budowa społeczności są uznane za konieczne dla przełamania ryzyka niskiego zainteresowania, lecz pozostają poza zautomatyzowanym zakresem oprogramowania MVP.

4.6 Model działania platformy

RacePortal funkcjonuje jako platforma dwustronna: organizatorzy dostarczają podaż wydarzeń, a kierowcy generują popyt na starty. Administrator pełni rolę gwaranta jakości katalogu poprzez akceptację wydarzeń i ról. Gość korzysta z efektu sieciowego katalogu bez bariery rejestracji, co obniża koszt wejścia informacyjnego.

W fazie MVP platforma **nie** pełni roli pośrednika płatniczego. Brak integracji z bramką PSP (np. Przelewy24, Stripe) jest decyzją świadomą, potwierdzoną w dokumentacji zakresowej i w kodzie. Dla wydarzeń płatnych system obsługuje statusy zgłoszenia (m.in. `PENDING`, `ACCEPTED`, `CONFIRMED`, `CANCELED`) oraz dołączenie dowodu wpłaty — jako adres URL albo ścieżka pliku w magazynie uploadów — natomiast rozliczenie finansowe odbywa się poza platformą (np. przelew na dane wskazane przez organizatora). Strumienie przychodów opisane w BMC (reklamy, płatna promocja wydarzeń, prowizje od zapisów) pozostają perspektywą rozwojową i nie są zaimplementowane.

Model darmowego dostępu w MVP dotyczy zarówno przeglądania katalogu, jak i korzystania z narzędzi kierowcy oraz panelu organizatora. Oznacza to, że weryfikacja wartości produktu na obecnym etapie opiera się na użyteczności operacyjnej, a nie na monetyzacji.

4.7 Realistyczność wdrożenia i dalsze perspektywy

Realistyczność wdrożenia została potwierdzona na dwóch poziomach. Poziom deweloperski obejmuje uruchomienie pełnego stosu przez Docker Compose (web, API, MySQL, kopia zapasowa) wraz z kontami seed służącymi do demonstracji ról. Poziom produkcyjny obejmuje wdrożenie na Oracle Cloud Always Free (`VM.Standard.A1.Flex`, 2 OCPU, 12 GB RAM, region Frankfurt) z udostępnieniem HTTPS poprzez Cloudflare Tunnel do nginx na porcie lokalnym maszyny. TLS i DNS są po stronie Cloudflare; aplikacja pracuje w układzie jednego originu (`/api` proxy do Spring Boot), co upraszcza konfigurację względem wcześniejszego wariantu GCP + Cloudflare Pages.

Założona w DZW skala pilotażowa (rzędu 1–5 tys. użytkowników miesięcznie, profil odczytowy do ok. 50 RPS) jest spójna z charakterem niskokosztowej infrastruktury dydaktycznej. Formalny test obciążenia przy 10k rekordach / 50 RPS nie został domknięty w MVP i pozostaje ograniczeniem odbiorczym, jawnie wskazanym w karcie projektu.

Dalsze perspektywy — zgodnie z dokumentacją produktową i materiałami BMC — obejmują m.in. integrację PSP, rozbudowę powiadomień push, publikację aplikacji mobilnej w sklepach, eksport list startowych, waitlistę/watchlistę, kalendarz iCal oraz ewentualną monetyzację poprzez oznaczone treści sponsorowane. Elementy te należy traktować jako roadmapę, a nie jako stan bieżącej implementacji. Szczegóły architektury fizycznej przedstawiono w rozdz. 8.3.

4.8 Kontekst prawny i etyczny

System przetwarza dane niezbędne do obsługi konta i procesów biznesowych: identyfikator/nazwę użytkownika, adres e-mail, role, dane pojazdów w garażu, zgłoszenia oraz dane wydarzeń wprowadzane przez organizatorów. Token sesji (JWT) jest przechowywany po stronie klienta. W interfejsie udostępniono politykę prywatności (`/privacy`) oraz regulamin (`/terms`); rejestracja wymaga akceptacji obu dokumentów. Przy tworzeniu wydarzenia organizator potwierdza przetwarzanie danych zgodnie z polityką prywatności (RODO) wyłącznie w celu publikacji i obsługi zgłoszeń. Kanałem kontaktowym w sprawach ochrony danych jest adres `rodo@raceportal.pl`. Użytkownik może zamknąć własne konto w ustawieniach (fraza potwierdzająca po stronie API).

Pełna zgodność procesowa (m.in. DPIA, sformalizowana polityka retencji, procedury realizacji żądań RODO w organizacji operatorskiej) wykracza poza zakres demonstracyjnego MVP i pozostaje obowiązkiem ewentualnego wdrożenia produkcyjnego o większej skali.

W warstwie etycznej dokumentacja SAI przyjmuje zasady: transparentności prezentacji wyników (brak ukrytego rankingu reklamowego w MVP), rezygnacji z mechanizmów uzależniających, dążenia do inkluzywności informacyjnej (wydarzenia o różnym progu wejścia) oraz przeciwdziałania dual-use (ryzyko publikacji nielegalnych wyścigów ulicznych). Mitigacją jest weryfikacja organizatorów i moderacja wydarzeń przez administratora. Odpowiedzialność za treść merytoryczną wydarzeń spoczywa na zweryfikowanych organizatorach w ramach regulaminu, natomiast odpowiedzialność za poprawność mechanizmów platformy — na zespole utrzymującym system.

Rozdział 5

Analiza SWOT

5.1 Mocne strony (Strengths)

- Pełny przepływ ról: gość, kierowca, organizator, administrator.
- Trzy widoki katalogu na wspólnych filtrach (lista / mapa / kalendarz).
- Konteneryzacja i seed kont demonstracyjnych ułatwiają prezentację.
- Testy automatyczne (API, web E2E, unit mobile / api-types).
- Wspólne typy API ograniczają dryf web $\leftrightarrow$ mobile.

5.2 Słabe strony (Weaknesses)

- Brak prawdziwego uploadu plików i bramki płatności.
- Mobilka bez logowania i garażu (wizytówka).
- Zależność od publicznych serwisów map/trasy (CARTO/OSRM).
- Częściowe rozjazdy względem wewnętrznej Specyfikacji Formularzy.
- Brak formalnego testu obciążenia 10k rekordów / 50 RPS.

5.3 Szanse (Opportunities)

- Rozszerzenie o PSP, waitlistę, watchlistę i eksport list startowych.
- Integracje z kalendarzami (iCal) i powiadomienia push.
- Budowa społeczności wokół torów i organizatorów regionalnych.

5.4 Zagrożenia (Threats)

- Konkurencja ze strony grup społecznościowych i dedykowanych systemów zapisów.
- Zmiany limitów / dostępności publicznych API map i routingu.
- Wymogi RODO i odpowiedzialność za dane osobowe przy skali produkcyjnej.

Rozdział 6

Planowanie projektu

6.1 Cele projektu

Dostarczenie działającego MVP katalogu i zapisów z rolami użytkowników, mapą, mailami, mobilną wizytówką oraz dokumentacją i testami umożliwiającymi odbiór / prezentację.

6.2 Zakres projektu

Zgodnie z listą funkcji MVP 1–15 (katalog, filtry, detal, admin, konto, garaż, zgłoszenia, organizator, baza, maile, mapa, trasa, archiwum, mobile) — szczegóły w dokumentacji projektowej `docs/MVP.md`.

6.3 Metodyka pracy

Iteracyjne dostarczanie funkcji w monorepo, z bieżącą dokumentacją zmian (`docs/changes.md`) oraz weryfikacją testami po istotnych zmianach.

6.4 Organizacja zespołu

Zespół pięcioosobowy pracuje we wspólnym repozytorium Git, z podziałem na warstwy web / API / mobile / dokumentacja / diagramy. Szczegółowy opis wkładu — rozdz. 17.

6.5 Harmonogram pracy

Prace obejmują m.in. prototyp Figma Make, scaffold UI, implementację backendu Spring, parity funkcji web, testy, uproszczenie mobile do wizytówki, wdrożenie Oracle + Cloudflare Tunnel oraz domykanie dokumentacji odbiorczej.

6.6 Infrastruktura i środowisko pracy

Docker Desktop, Node.js (web/mobile), JDK/Maven (API w kontenerze), Expo / iOS Simulator lub Android Emulator, Playwright, GitHub, Cloudflare Tunnel (`cloudflared`), Oracle Cloud VM.

6.7 Analiza ryzyka projektowego

6.7.1 Analiza zagrożeń

- Niespójność wymagań formularzy vs implementacja — mitigacja: rejestr luk (`spec-conformity.md`).
- Awaria zewnętrznego OSRM/CARTO — mitigacja: graceful fallback komunikatów, możliwość wymiany providera.
- Dryf typów web/mobile — mitigacja: `packages/api-types`.
- Opóźnienia wynikające z rozrostu zakresu — mitigacja: świadome odłożenie galerii/P-SP/store.

Rozdział 7

Analiza wymagań

7.1 Źródła i sposób pozyskania wymagań

Pitch / zakres MPC, Specyfikacja Formularzy, prototyp Figma Make, iteracje implementacyjne, scenariusze prezentacji oraz przeglądy kodu.

7.2 Wymagania ogólne i dziedzinowe

System ma obsługiwać polski kontekst motorsportu (kategorie, lokalizacje, role) i umożliwiać kompletny happy-path zapisu na wydarzenie.

7.3 Wymagania funkcjonalne

Poniżej przykładowe karty wymagań (środowisko / makro szablonu PJATK). Pełna lista MVP pozostaje w `docs/MVP.md`.

KARTA WYMAGANIA			
Identyfikator:	FR-01	Priorytet:	Wysoki
Nazwa:	Przeglądanie katalogu wydarzeń		
Opis:	System umożliwia przeglądanie wydarzeń w widokach lista, mapa i kalendarz ze wspólnymi filtrami.		
Udziałowiec:	Gość, Kierowca (tab. 4.1)		
Wymagania powiązane:	UI katalogu — rozdz. 11; API `/api/events` — rozdz. 8		

Tabela 7.1: Przeglądanie katalogu wydarzeń

1. Przeglądanie wydarzeń (lista, mapa, kalendarz) z filtrami.
2. Podgląd szczegółów wydarzenia.
3. Rejestracja, logowanie, reset hasła; opcjonalnie Google Sign-In.
4. Zarządzanie profilem i garażem.

KARTA WYMAGANIA			
Identyfikator:	FR-02	Priorytet:	Wysoki
Nazwa:	Zgłoszenie udziału w wydarzeniu		
Opis:	Zalogowany kierowca wybiera pojazd z garażu, system waliduje wymagania (`carMatch`) i tworzy zgłoszenie ze statusem oraz powiadomieniem e-mail.		
Udziałowiec:	Kierowca (tab. 4.1)		
Wymagania powiązane:	Garaż, mail, RBAC — rozdz. 10, 13		

Tabela 7.2: Zgłoszenie udziału w wydarzeniu

KARTA WYMAGANIA			
Identyfikator:	FR-03	Priorytet:	Wysoki
Nazwa:	Panel organizatora i moderacja admina		
Opis:	Organizator zarządza własnymi wydarzeniami i zgłoszeniami; administrator akceptuje/odrzuca wydarzenia oraz wnioski o rolę ORGANIZER.		
Udziałowiec:	Organizator (tab. 4.2), Administrator (tab. 4.3)		
Wymagania powiązane:	Projekt paneli — rozdz. 8; realizacja — rozdz. 11		

Tabela 7.3: Panel organizatora i moderacja admina

5. Składanie i anulowanie zgłoszeń; podgląd statusów.
6. Panel organizatora (CRUD wydarzeń, obsługa zgłoszeń).
7. Panel administratora (moderacja, role, wnioski org.).
8. Powiadomienia e-mail.
9. Archiwum wydarzeń.
10. Aplikacja mobilna — katalog + CTA do weba.

7.4 Wymagania niefunkcjonalne

Dostępność lokalna przez Docker; responsywność weba; podstawowa wydajność list (paginacja, cache); obserwowalność (`/api/health`); dokumentacja uruchomienia. Docelowy test obciążenia 10k rekordów / 50 RPS pozostaje poza domkniętym MVP (por. rozdz. 18).

7.5 Wymagania bezpieczeństwa

Uwierzytelnianie JWT; RBAC; walidacja wejścia; hasła hashowane; nagłówki bezpieczeństwa nginx (CSP, X-Frame-Options); rate limiting; minimalizacja danych w UI.

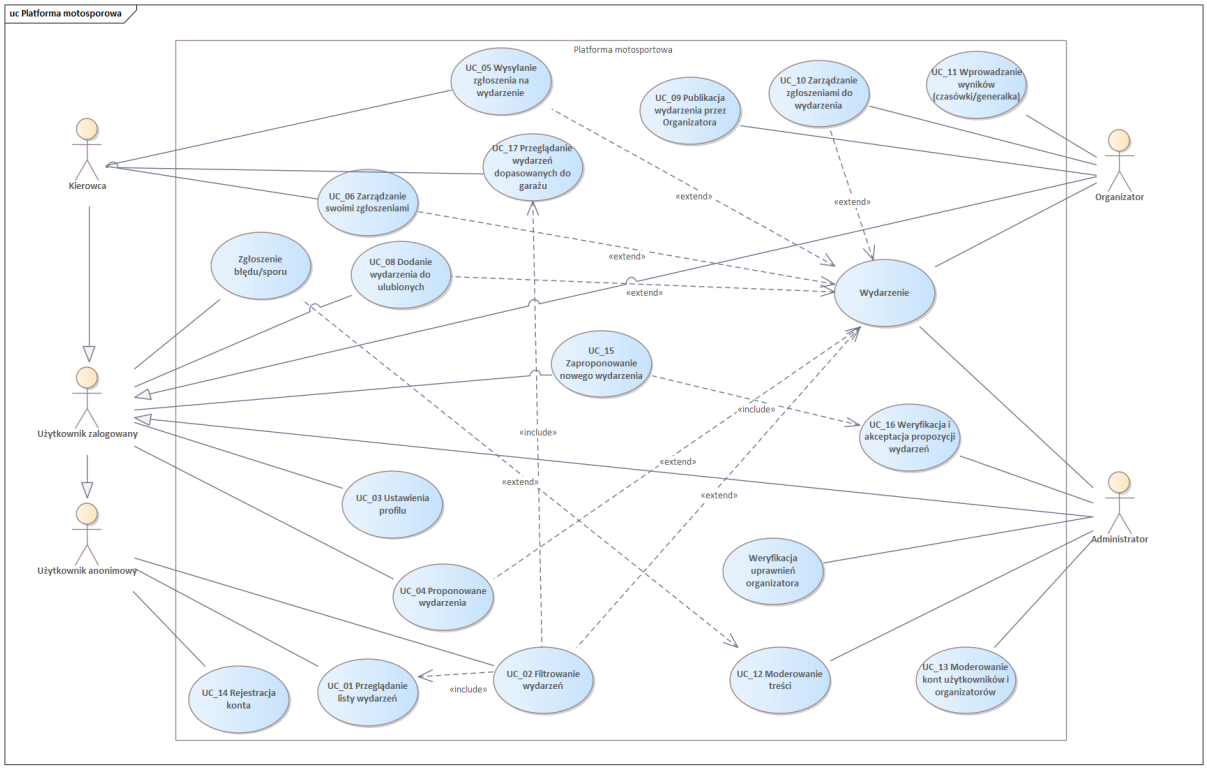
7.6 Wymagania interfejsu użytkownika

Ciemny motyw motorsportowy z akcentem marki; czytelne stany puste/błędu; spójność paneli ról; mobilka z prostą nawigacją katalogu. Punkt wyjścia UI: prototyp Figma Make (rozdz. 8.6).

7.7 Diagramy UML

7.7.1 Diagram przypadków użycia

Wczesny diagram przypadków użycia (materiały projektowe) przedstawia rys. 7.1.



Rysunek 7.1: Diagram przypadków użycia — Platforma motorsportowa (wersja koncepcyjna).

7.7.2 Diagram sekwencji: rejestracja i logowanie

7.7.3 Diagram sekwencji: zapis na wydarzenie

7.7.4 Diagram sekwencji: zarządzanie garażem

Uwaga: diagram klas / model danych przeniesiono do projektu systemu — podrozdział 8.4.1.

Rozdział 8

Projekt systemu

8.1 Założenia architektoniczne

Architektura trójwarstwowa: klienty (web, mobile) → API REST → MySQL. Statyczny frontend za nginx z proxy `/api`.

8.2 Architektura logiczna systemu

Moduły domenowe: użytkownicy/auth, wydarzenia, zgłoszenia, garaż, organizator, administracja, mapy/trasa, poczta.

8.3 Architektura fizyczna i wdrożeniowa

Kontenery lokalne: `web`, `api`, `mysql`, `backup`. Mobile poza Compose. Maile — SMTP Mailbux (poza Compose; `SMTP_PASSWORD` w `.env`).

Wdrożenie produkcyjne: maszyna Oracle Always Free (ARM) z Docker Compose; publiczny dostęp przez **Cloudflare Tunnel** (`cloudflared`) do nginx na porcie `:8081`. Domena `https://raceportal.pl` jest w tym samym koncie Cloudflare — TLS/HSTS i DNS po stronie Cloudflare, bez hostowania frontu na Cloudflare Pages.

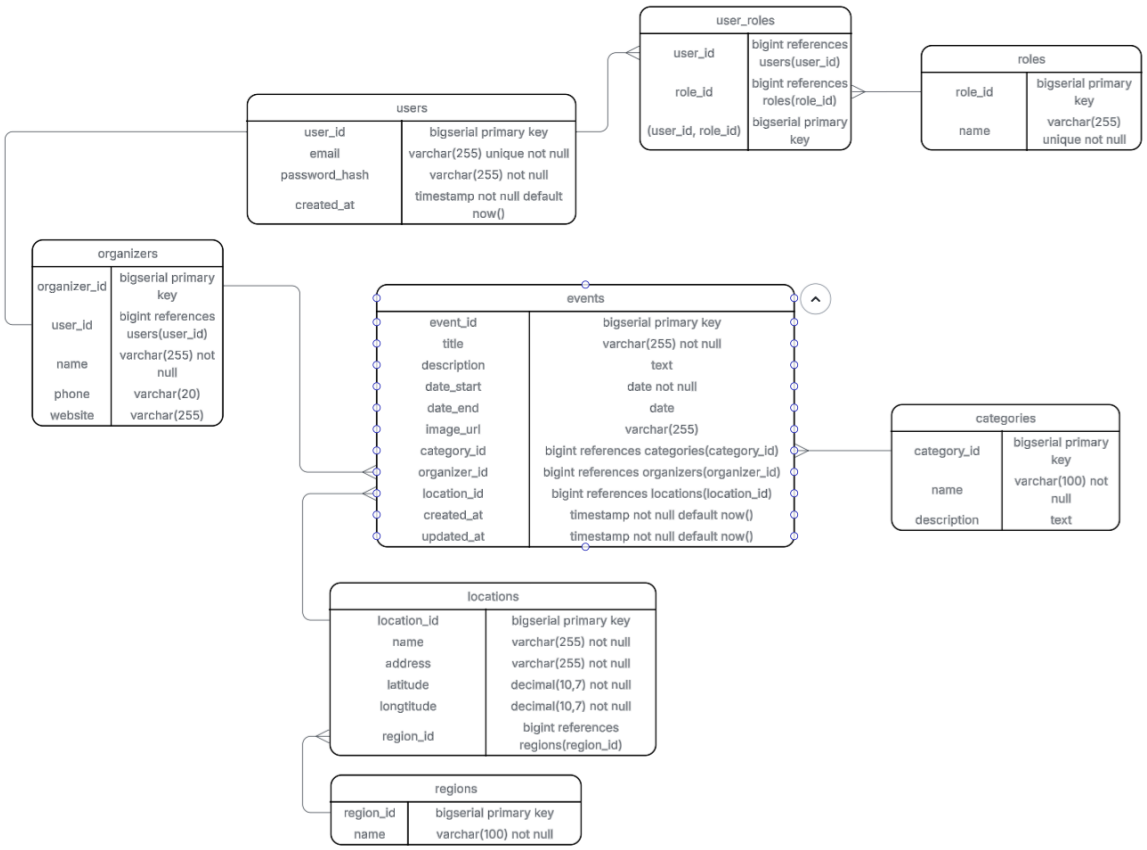
8.4 Projekt bazy danych

Encje m.in.: użytkownik, pojazd, wydarzenie, zgłoszenie, wniosek organizatora, **tor** (katalog PL seedowany przy starcie API, odczyt `GET /api/tracks`); statusy wydarzeń i zgłoszeń; współrzędne geograficzne. Wspólne presety torów dla UI: `TRACK_PRESETS` w `packages/api-types`.

8.4.1 Diagram klas

Rysunek 8.1 przedstawia wczesny model bazy danych (ERD) stanowiący punkt wyjścia projektu klas / tabel domenowych. W implementacji Spring Data JPA model został uproszczony

i dostosowany do MVP (m.in. role jako enum użytkownika, lokalizacja osadzona w wydarzeniu).



Rysunek 8.1: Diagram klas / model danych (ERD) — wersja koncepcyjna projektu.

8.5 Projekt interfejsu REST API

Zasoby `/api/auth/*`, `/api/events`, `/api/events/markers`, `/api/registrations`, `/api/garage`, `/api/admin/*`, `/api/organizer/*`, `/api/maps/route`, `/api/health`.

8.6 Projekt warstwy webowej

Routing React Router; AuthContext; strony katalogu, detalu, dashboardu, garażu, paneli; komponent mapy Leaflet.

8.6.1 Prototyp UI w Figma Make

Wyjściowym projektem interfejsu był plik Figma Make **Advanced UI/UX for RACEPORTAL** (ciemny motyw motorsportowy, akcent marki, Orbitron / typografia display, komponenty zbli-

żone do shadcn/ui). Prototyp obejmował m.in. stronę główną, katalog, logowanie/rejestrację, dashboard oraz szkice paneli organizatora i administratora.

Kod frontendu został przeniesiony z Figma Make do lokalnego monorepo (Vite/React), a następnie podpięty pod realne API (zamiast mocków / localStorage). Wczesna wizja produktu i ról — jeszcze przed pełną implementacją — jest też widoczna na diagramach koncepcyjnych (rys. 7.1, 4.1).

W toku realizacji UI dopracowano pod kątem responsywności, stanów pustych/błędów, PWA (manifest, service worker, dolna nawigacja mobilna) oraz spójności z tokenami marki (`-race-accent`).

8.7 Projekt aplikacji mobilnej

Stack Expo; nawigacja stack Eventy; publiczny klient HTTP; `EXPO_PUBLIC_API_URL` / `EXPO_PUBLIC_WEB_UR` CTA `Linking` do `/wydarzenia/:id`.

8.8 Projekt mechanizmów bezpieczeństwa

JWT w nagłówku Authorization (web); role na endpointach; walidacja DTO; CORS; CSP nginx.

Rozdział 9

Realizacja

9.1 Organizacja implementacji

Implementacja iteracyjna w monorepo, z dokumentowaniem zmian i utrzymywaniem środowiska Compose jako źródła prawdy uruchomieniowej.

9.2 Podział systemu na komponenty

9.2.1 Backend

Spring Boot: kontrolery, serwisy, repozytoria JPA, seed (`DataInitializer`), joby (np. archiwizacja).

9.2.2 Frontend webowy

Vite + React; integracja z API; stany ról i formularze dziedzinowe.

9.2.3 Aplikacja mobilna

Expo — katalog publiczny bez auth.

9.3 Integracja komponentów

Wspólny kontrakt JSON; typy w `api-types`; nginx reverse proxy `/api` w środowisku Compose.

9.4 Konteneryzacja i uruchamianie środowiska

`docker compose up -build -d`; weryfikacja `/api/health`; Expo: `cd mobile && npm start`. Publiczny dostęp produkcyjny: Cloudflare Tunnel → nginx `:8081`.

Rozdział 10

Szczegóły implementacji backendu

10.1 Struktura projektu backendowego

Pakiet `pl.raceportal` — warstwy controller / service / repository / dto / security.

10.2 Model domenowy i encje

Użytkownik (role), Car, Event, Registration oraz powiązane statusy i flagi wymagań wydarzenia.

10.3 Rejestracja użytkowników i logowanie

Rejestracja z aktywacją linkiem `/verify-email/{token}` (`PUBLIC_BASE_URL`; query `?token=` jako fallback); logowanie hasłem; opcjonalna wymiana Google idToken → JWT; rate limit prób w API (`AttemptRateLimiter`).

10.4 Zarządzanie profilami użytkowników

Aktualizacja danych profilu, hasła, deklaracji (m.in. PZM, prawo jazdy B), pól adresu / Instagram.

10.5 Obsługa wydarzeń

Listowanie z filtrami i paginacją; markery mapy; CRUD po stronie organizatora; moderacja admina; archiwum.

10.6 Obsługa zgłoszeń

Tworzenie zgłoszenia z wyborem pojazdu; walidacja wymagań; zmiana statusów; anulowanie; maile transactional.

10.7 Obsługa garażu

CRUD pojazdów; reguły clear-on-edit pól liczbowych; powiązanie z kategorią wydarzenia.

10.8 Panel administratora i organizatora

Endpointy i reguły RBAC dla moderacji i zarządzania zgłoszeniami / wydarzeniami.

10.9 Integracja z bazą danych

Spring Data JPA + MySQL 8; konfiguracja przez zmienne środowiskowe Compose.

10.10 Obsługa wyjątków i walidacja

Bean Validation; odpowiedzi z polem `details`; mapowanie na komunikaty UI.

Rozdział 11

Szczegóły implementacji frontendu webowego

11.1 Struktura aplikacji webowej

Katalog `web/src/app`: `pages/`, `components/`, `context/`, `lib/`.

11.2 Strona główna i katalog wydarzeń

Widoki Lista / Mapa / Kalendarz; debounce filtrów; „Wyczyść filtry”; paginacja listy vs markery overview.

11.3 Widok szczegółów wydarzenia

Opis, wymagania, zapis z wyborem auta, `carMatch`, mapa i trasa.

11.4 Logowanie i rejestracja

Formularze auth; weryfikacja kodu; reset hasła; opcjonalny Google Sign-In.

11.5 Dashboard użytkownika

Zgłoszenia użytkownika i szybkie informacje o nadchodzących startach.

11.6 Widok garażu

Formularz pojazdu, walidacja pól wymaganych, edycja i usuwanie (z ograniczeniami przy otwartym zgłoszeniu).

11.7 Panel organizatora

Tworzenie/edycja wydarzeń, lista zgłoszeń, zmiana statusów.

11.8 Panel administratora

Moderacja wydarzeń, wnioski organizatora (zaakceptuj / odrzuć → rola `ORGANIZER`), zarządzanie rolami, zgłoszenia błędów. Formularz wniosku użytkownika: `/zostan-organizatorem`.

11.9 Mapa i elementy lokalizacyjne

Leaflet + kafelki CARTO; pickery lokalizacji; proxy trasy `/api/maps/route`.

11.10 Responsywność i doświadczenie użytkownika

Układ dostosowany do desktop/mobile web; motyw z akcentem `-race-accent`.

Rozdział 12

Szczegóły implementacji aplikacji mobilnej

12.1 Założenia aplikacji mobilnej

Mobilka ma wspierać odkrywanie wydarzeń, a nie dublować pełnego klienta webowego.

12.2 Architektura rozwiązania mobilnego

Expo SDK, React Navigation (stack Eventy), ThemeProvider, cienki klient API bez JWT.

12.3 Lista wydarzeń i widok szczegółów

Filtry publiczne; przełączanie Lista / Kalendarz / Mapa; detal z metadanymi wydarzenia.

12.4 Nawigacja i interfejs użytkownika

Pojedynczy stack: `EventsList` → `EventDetail`; dark UI spójny z marką.

12.5 Integracja z API

Publiczne GET (events, markers, health); konfiguracja hosta API (localhost / LAN / Android emulator).

12.6 Przekierowanie do wersji webowej

Przycisk „Zapisz się na stronie” — `Linking.openURL(webEventUrl(id))`.

12.7 Ograniczenia wersji mobilnej

Brak auth, garażu, paneli ról i archiwum w aplikacji; zależność od dostępności weba przy konwersji zapisu.

Rozdział 13

Bezpieczeństwo systemu

13.1 Założenia bezpieczeństwa

Ochrona kont i danych zgłoszeń na poziomie ASVS L1 adekwatnym do MVP demonstracyjnego.

13.2 Klasyfikacja danych

Dane konta i profilu; dane pojazdów; dane zgłoszeń; treści wydarzeń (częściowo publiczne).

13.3 Uwierzytelnianie użytkowników

Hasło + JWT; opcjonalnie Google; brak sesji serwerowej klasycznej.

13.4 Autoryzacja i kontrola dostępu RBAC

Role USER / ORGANIZER / ADMIN egzekwowane w API i bramkach UI.

13.5 Wykorzystanie tokenów JWT

Token przechowywany po stronie klienta web; dołączany do żądań chronionych.

13.6 Walidacja danych wejściowych

Walidacja DTO po stronie API; walidacja formularzy UI; komunikaty pól `details`.

13.7 Ochrona zasobów użytkowników

Dostęp do garażu i zgłoszeń wyłącznie właściciela (oraz uprawnionych ról w panelach).

13.8 Bezpieczeństwo komunikacji

W dev HTTP lokalne; w prod TLS terminowany na Cloudflare Tunnel przed nginx; CSP / X-Frame-Options na nginx.

13.9 Analiza zagrożeń

Uwierzytelnianie, IDOR, XSS, CSRF w kontekście SPA, brute-force logowania (rate limit).

13.10 Analiza ryzyka

Ryzyko akceptowalne dla środowiska demo; produkcja wymaga twardego sekretu JWT, HTTPS i polityki haseł zsynchronizowanej backend↔UI.

13.11 Ograniczenia przyjętych zabezpieczeń

Brak 2FA; Google OAuth opcjonalny; brak WAF; publiczny OSRM bez SLA.

Rozdział 14

Testowanie i weryfikacja systemu

14.1 Strategia testowania

Piramida: unit (api-types, mobile client) → integracyjne API (MockMvc) → E2E Playwright (web + preview mobile).

14.2 Środowisko testowe

Docker Compose + opcjonalnie Expo web `:8082`; dokumentacja `docs/testy/TESTY.md`.

14.3 Testy jednostkowe

Vitest: `packages/api-types`, `mobile/tests`.

14.4 Testy integracyjne backendu

JUnit / MockMvc — scenariusze auth, eventy, garaż, zgłoszenia, RBAC, trasa.

14.5 Testy End-to-End aplikacji webowej

Playwright: ścieżki katalogu, auth, garaż, role (viewport desktop/mobile).

14.6 Testy aplikacji mobilnej

Unit URL API/WEB; E2E gość (lista, widoki, detal, CTA) na Expo web.

14.7 Testy mechanizmów bezpieczeństwa

Scenariusze braku dostępu do paneli obcych ról; walidacja wejścia.

14.8 Automatyzacja uruchamiania testów

Skrypty npm / `scripts/`; raporty w `docs/testy/wyniki/`.

14.9 Analiza wyników testów

Werdykty przebiegów odnotowywane w dokumentacji wyników; regresie łapane przed prezentacją.

14.10 Ograniczenia procesu testowania

Brak pełnych testów natywnych (Detox/Maestro); **brak formalnego load testu 10k rekordów / 50 RPS** w MVP — ograniczenie omówione w podsumowaniu.

Rozdział 15

Prezentacja systemu

15.1 Prezentacja głównych funkcjonalności

Demonstracja katalogu (3 widoki), zapisu kierowcy z `carMatch`, paneli org/admin oraz mobile CTA — zgodnie z `docs/scenariusz-prezentacji.md`.

15.2 Scenariusz użycia z perspektywy kierowcy

Login seed → garaż → filtr/katalog → zapis → dashboard statusów.

15.3 Scenariusz użycia z perspektywy organizatora

Login org → wydarzenie (presety torów) → zgłoszenia → zmiana statusu → podgląd maila na skrzynce (Mailbux).

15.4 Scenariusz użycia z perspektywy administratora

Login admin → moderacja wydarzenia / wniosku organizatora / ról.

15.5 Podsumowanie działania systemu

System realizuje spójny obieg informacji o wydarzeniach i zapisach w środowisku uruchamialnym lokalnie oraz pod adresem produkcyjnym za Cloudflare Tunnel.

Rozdział 16

Nakład pracy

16.1 Liczba godzin przeznaczona na realizację projektu

16.2 Podział zadań w zespole

Skrót obszarów — rozdz. 17. Szczegółowa tabela godzin: TODO.

16.3 Narzędzia wspierające zarządzanie pracą

Git/GitHub, dokumentacja Markdown w `docs/`, Docker, Cursor/IDE, Playwright, Figma Make, Cloudflare Dashboard.

16.4 Problemy organizacyjne i ich rozwiązanie

M.in. rozrost zakresu mobile — decyzja o wizytówce; luki Spec — rejestr `spec-conformity.md`; stabilizacja testów względem seedu/archiwizacji; uproszczenie hostingu (Oracle + Tunnel zamiast GCP Pages).

Rozdział 17

Wkład własny

Rozdział opisuje, kto odpowiadał za które obszary prac projektowych i implementacyjnych (imię i nazwisko). Podział odzwierciedla faktyczny udział zespołu; największy wkład merytoryczny i projektowy w warstwie produktu/UI wniosła Natalia Otrombke.

17.1 Natalia Otrombke

- projekt UI/UX w Figma Make (**Advanced UI/UX for RACEPORTAL**) — ekrany katalogu, auth, dashboard, paneli;
- przeniesienie i dopracowanie frontendu React (motyw, layout, spójność marki, kluczowe ścieżki UX);
- udział w definicji flow użytkownika (gość $\rightarrow$ kierowca $\rightarrow$ organizator) oraz w materiałach koncepcyjnych ról.

17.2 Oliwier Kasprowicz

- prace analityczne i dokumentacyjne (wymagania, prezentacje zespołowe);
- wsparcie przy diagramach i scenariuszach użycia;
- udział w przeglądach zakresu MVP i odbiorze funkcjonalności.

17.3 Michał Gutowski

- diagramy i materiały projektowe (m.in. rich picture, przepływy);
- wsparcie dokumentacji i testów scenariuszy prezentacji;
- udział w integracji i weryfikacji ścieżek użytkownika.

17.4 Miłosz Parkitny

- modelowanie i diagramy (m.in. przypadki użycia, podział funkcjonalności, model danych);
- wsparcie analizy interesariuszy i harmonogramów;
- udział w pracach implementacyjnych / poprawkach w repozytorium.

17.5 Wojciech Wronisz

- backend Spring Boot (API, JWT/RBAC, encje, seed, maile);
- Docker Compose, wdrożenie produkcyjne Oracle + **Cloudflare Tunnel**, Mailbux, PWA;
- dokumentacja uruchomienia i odbioru (`docs/`), testy API / E2E, integracja web $\leftrightarrow$ API.

Rozdział 18

Podsumowanie

18.1 Największe wyzwania i sposób ich rozwiązania

Spójność domeny zapisów i garażu, RBAC, mapa/trasa oraz utrzymanie monorepo web+API+mobile. Po stronie infrastruktury — uproszczenie hostingu do jednej VM z tunelowaniem Cloudflare zamiast rozdzielonego stacku Pages/Worker.

18.2 Zmiany i adaptacje w trakcie realizacji

Przejście mobile z parity na wizytówkę; doprecyzowanie formularzy; wspólny pakiet typów; migracja z eksperymentów GCP/Pages na Oracle + Cloudflare Tunnel.

18.3 Ograniczenia końcowej wersji systemu

Brak PSP i uploadu plików; mobilka bez konta; limity testów wydajnościowych i produkcyjnego hardeningu.

18.3.1 Ograniczenia wydajnościowe (RPS)

W wymaganiach jakościowych MVP zapisano docelowy test obciążenia: **ok. 10 000 rekordów** w katalogu oraz **ok. 50 RPS** przy akceptowalnym P95 odpowiedzi list/detalu. W oddawanej wersji:

- zastosowano mitigacje lokalne (paginacja listy, osobne markery mapy, indeksy / cache tam gdzie uzasadnione);
- **nie wykonano** formalnego load testu 10k/50 RPS (brak raportu RPS/P95 jako artefaktu odbioru);

- środowisko produkcyjne (Oracle Always Free + Cloudflare Tunnel) jest wystarczające do demonstracji i ruchu prezentacyjnego, ale nie zostało zweryfikowane pod kątem założonego progu 50 RPS.

Ograniczenie to jest świadome: priorytetem było domknięcie happy-path funkcjonalnego, bezpieczeństwa podstawowego (JWT/RBAC) i uruchamialności. Pomiar 10k/50 RPS pozostaje w backlogu jakościowym.

18.4 Możliwości dalszego rozwoju

Płatności, upload, watchlista, iCal, waitlista, eksport list startowych, store/PWA — por. `docs/pomysly-rozwo`
uzupełnienie testów obciążeniowych RPS.

18.5 Wnioski końcowe

RacePortal stanowi kompletne MVP demonstracyjne katalogu i zapisów motorsportowych z czytelnością architektury, testami funkcjonalnymi i ścieżką prezentacji dla odbiorcy akademickiego oraz biznesowego — przy jasno nazwanych lukach (PSP, upload, load test RPS).

Bibliografia

[1] „RacePortal — repozytorium projektu”. `https://github.com/Gucio-009/RacePortal`. Branch `wojtek`; dostęp: 2026-10-15.

Spis rysunków

4.1 Podział interesariuszy projektu — funkcjonalności dostępne dla ról użytkowników (materiał koncepcyjny). 30

7.1 Diagram przypadków użycia — Platforma motorsportowa (wersja koncepcyjna). 41

8.1 Diagram klas / model danych (ERD) — wersja koncepcyjna projektu. 43

Spis tabel

4.1 Karta udziałowca: Kierowca 30
4.2 Karta udziałowca: Organizator 31
4.3 Karta udziałowca: Administrator 31

7.1 Przeglądanie katalogu wydarzeń 39
7.2 Zgłoszenie udziału w wydarzeniu 40
7.3 Panel organizatora i moderacja admina 40

Załączniki

Płyta CD z następującą zawartością:

- *pliki projektowe* – pliki składające się na całość projektu
  - repozytorium kodu źródłowego wraz z instrukcją zbudowania i uruchomienia projektu
  - źródło pracy inżynierskiej.
- *Otrombke Natalia_Kasprowicz Oliwier_Gutowski Michał_Parkitny Miłosz_Wronisz Wojciech_praca pisemna* – katalog zawierający plik PDF z pracą inżynierską.